

DTS103TC

Data Structure and Algorithms

Lab 8: Graph theory

Dr. Qi Chen, Dr. Di Zhang, Dr. Md Maruf Hasan,
Dr. Xuechen Liu, Dr. Guangyu Ren and Dr. Bin Chen
School of AI and Advanced Computing

Learning outcome

- Graph representation (adjacency list, adjacency matrix)
- Breadth First Search
 - Exercise: Find Shortest Path
- Depth First Search
 - Exercise: Detecting Cycles in Undirected Graphs

Graph Representations

Basic representation

- Let $G = (V, E)$ a graph
 - $V = \{v_1, v_2, \dots, v_n\}$ the set of vertices
 - $E = \{e_1, \dots, e_m\}$ the set of edges
- Notes
 - $e = (a, b)$ is an edge from a to b with a, b in V
 - (b, a) is not necessarily in V , if G is directed

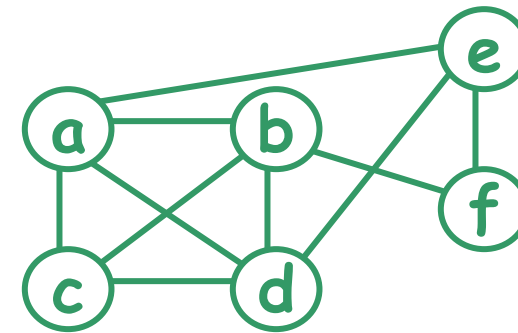
Basic representation

- Let $G = (V, E)$ a graph
 - $V = \{v_1, v_2, \dots, v_n\}$ the set of vertices
 - $E = \{e_1, \dots, e_m\}$ the set of edges

- Example

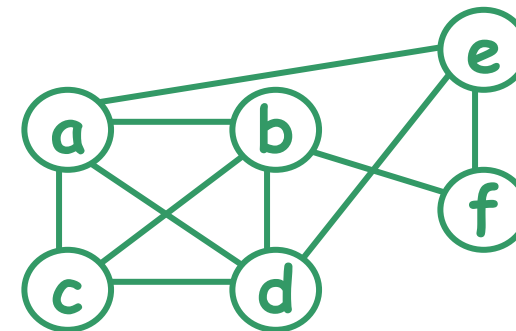
- $V = \{a, b, c, d, e, f\}$
- $E = \{(a, e), (a, b), (a, d), (a, c),$
 $(b, c), (b, d), (b, f),$
 $(c, d), (d, e), (e, f)\}$

(undirected notation)



Basic representation

- $V = \{a, b, c, d, e, f\}$
- $E = \{(a, e), (a, b), (a, d), (a, c), (b, c), (b, d), (b, f), (c, d), (d, e), (e, f)\}$
- $G = (V, E)$



```
# Define the set of vertices
```

```
V = ['a', 'b', 'c', 'd', 'e', 'f']
```

```
# Define the set of edges as tuples (undirected edges)
```

```
E = [  
    ('a', 'e'), ('a', 'b'), ('a', 'd'), ('a', 'c'),  
    ('b', 'c'), ('b', 'd'), ('b', 'f'),  
    ('c', 'd'), ('d', 'e'), ('e', 'f')  
]
```

```
# Graph G is defined as a tuple of (V, E)
```

```
G = (V, E)
```

```
print(V)
```

```
print(E)
```

```
print(G)
```

```
['a', 'b', 'c', 'd', 'e', 'f']
```

```
[('a', 'e'), ('a', 'b'), ('a', 'd'), ('a', 'c'), ('b', 'c'), ('b', 'd'), ('b', 'f'), ('c', 'd'), ('d', 'e'), ('e', 'f')]
```

```
[['a', 'b', 'c', 'd', 'e', 'f'], [('a', 'e'), ('a', 'b'), ('a', 'd'), ('a', 'c'), ('b', 'c'), ('b', 'd'), ('b', 'f'), ('c', 'd'), ('d', 'e'), ('e', 'f')]]
```

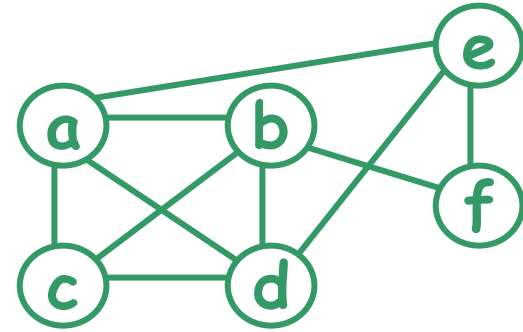
Adjacency list

```
# Directly define the adjacency list of the undirected graph
adj_list = {
    'a': ['e', 'b', 'd', 'c'],
    'b': ['a', 'c', 'd', 'f'],
    'c': ['a', 'b', 'd'],
    'd': ['a', 'b', 'c', 'e'],
    'e': ['a', 'd', 'f'],
    'f': ['b', 'e']
}

# Print adjacency list
print("Adjacency List:")
for vertex in adj_list:
    print(f"{vertex}: {adj_list[vertex]}")
```

Adjacency List:

```
a: ['e', 'b', 'd', 'c']
b: ['a', 'c', 'd', 'f']
c: ['a', 'b', 'd']
d: ['a', 'b', 'c', 'e']
e: ['a', 'd', 'f']
f: ['b', 'e']
```



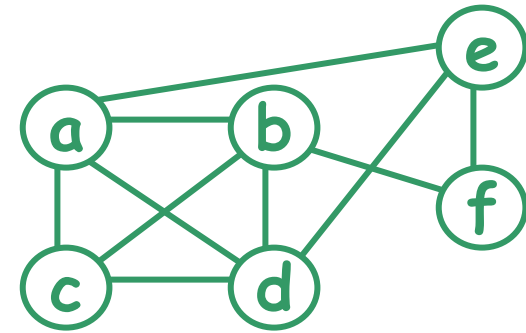
Adjacency matrix

```
# Adjacency matrix for the undirected graph
# a b c d e f
adj_matrix = [
    [0, 1, 1, 1, 1, 0], # a
    [1, 0, 1, 1, 0, 1], # b
    [1, 1, 0, 1, 0, 0], # c
    [1, 1, 1, 0, 1, 0], # d
    [1, 0, 0, 1, 0, 1], # e
    [0, 1, 0, 0, 1, 0]  # f
]

# Print the adjacency matrix with labels
print("Adjacency Matrix:")
print("    " + " ".join(V))
for i, row in enumerate(adj_matrix):
    print(f"V[{i}]: " + " ".join(map(str, row)))
```

Adjacency Matrix:

	a	b	c	d	e	f
a:	0	1	1	1	1	0
b:	1	0	1	1	0	1
c:	1	1	0	1	0	0
d:	1	1	1	0	1	0
e:	1	0	0	1	0	1
f:	0	1	0	0	1	0



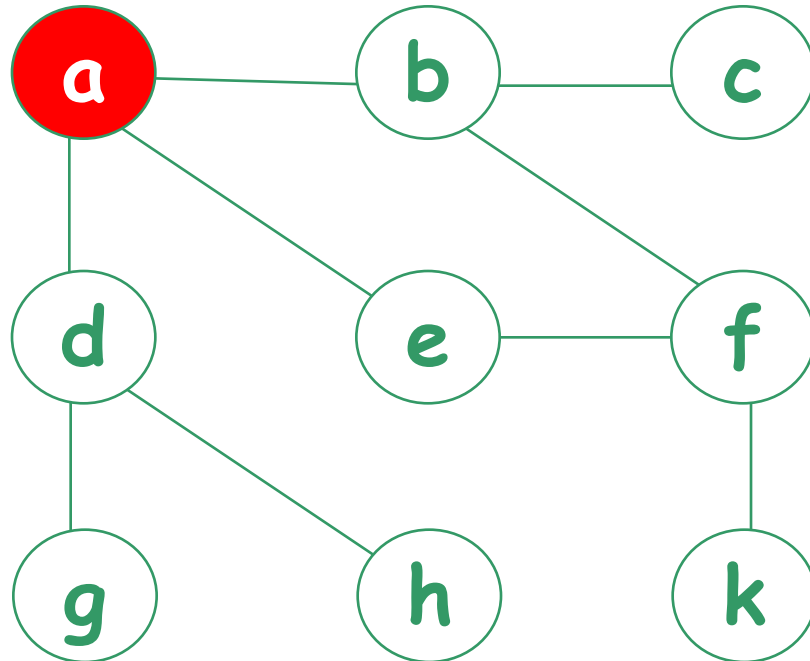
Breadth First Search (BFS)

Breadth First Search (BFS)

- All vertices at distance k from s are explored before any vertices at distance $k+1$.

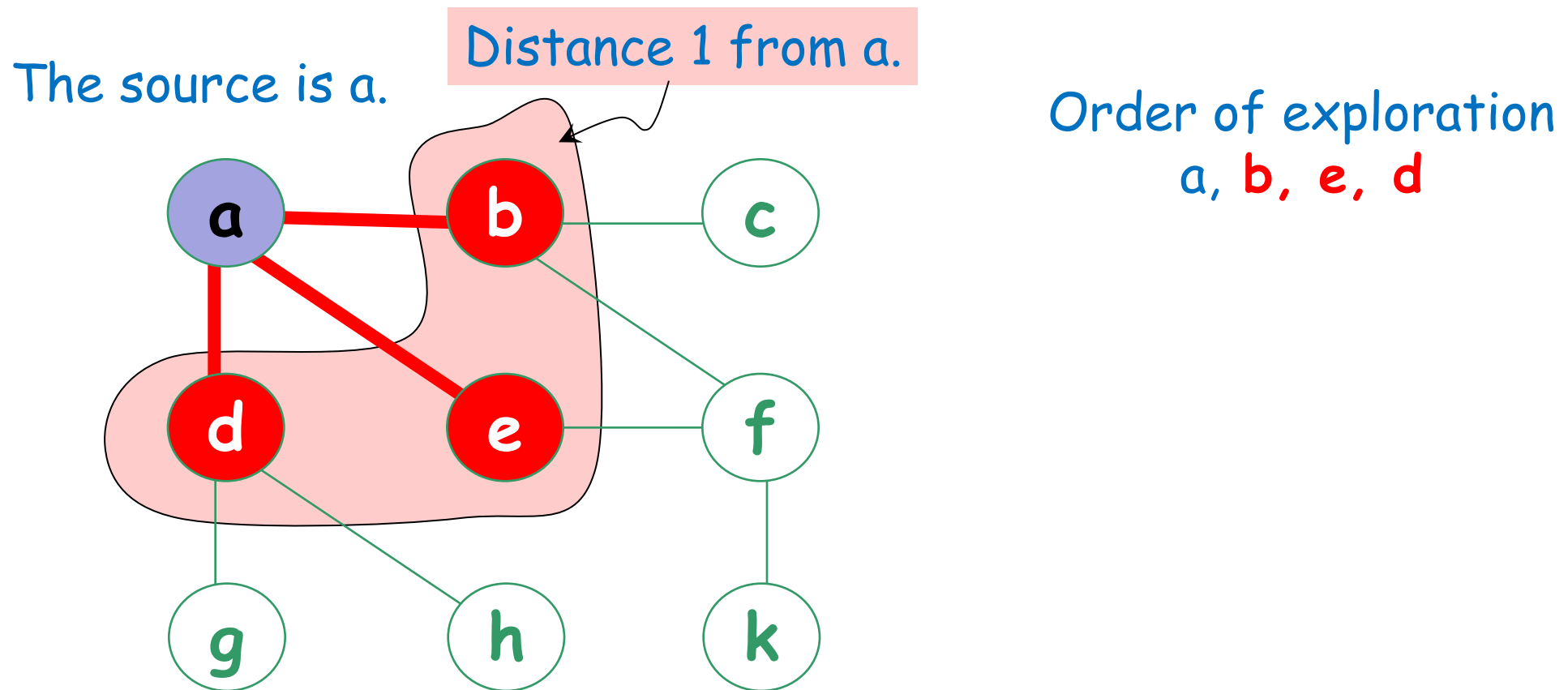
The source is a.

Order of exploration
a,



Breadth First Search (BFS)

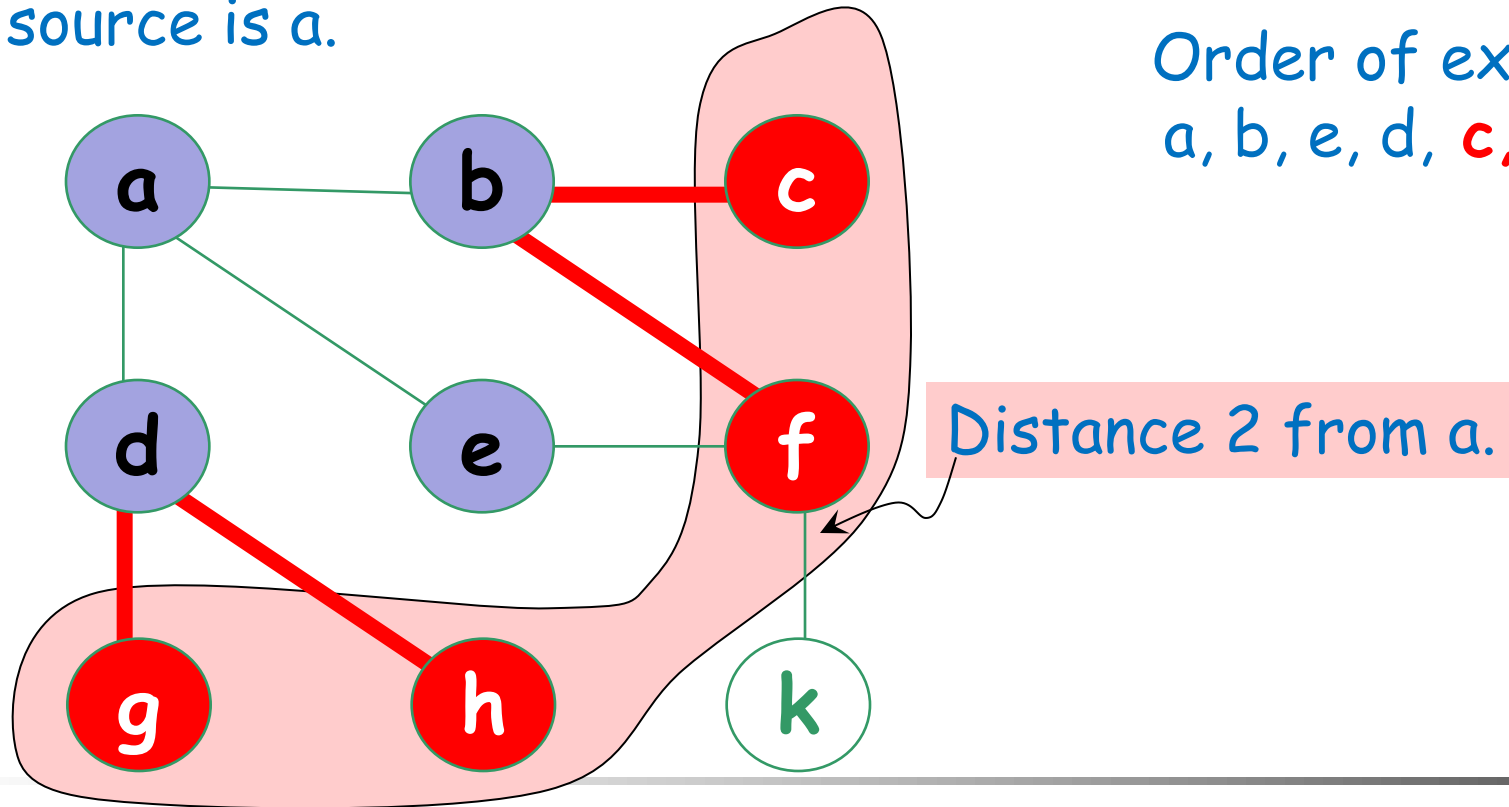
- All vertices at distance k from s are explored before any vertices at distance $k+1$.



Breadth First Search (BFS)

- All vertices at distance k from s are explored before any vertices at distance $k+1$.

The source is a.



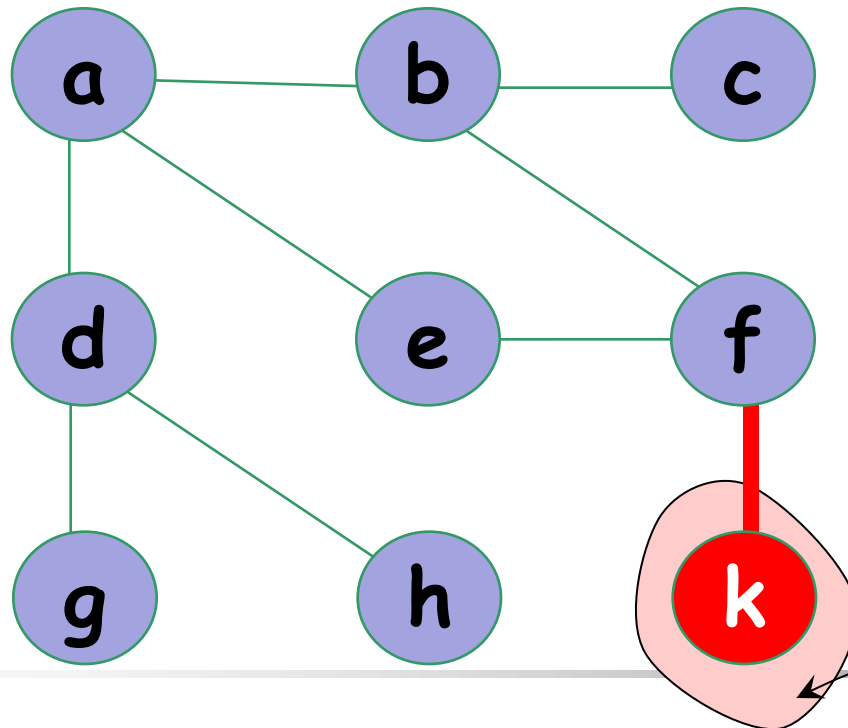
Order of exploration
a, b, e, d, **c**, **f**, **h**, **g**

Breadth First Search (BFS)

- All vertices at distance k from s are explored before any vertices at distance $k+1$.

The source is a.

Order of exploration
a, b, e, d, c, f, h, g, **k**



Distance 3 from a.

Breadth First Search (BFS)

- A simple algorithm for searching a graph.
- Given $G=(V, E)$, and a distinguished source vertex s , BFS systematically explores the edges of G such that
 - all vertices at distance k from s are explored before any vertices at distance $k+1$.

BFS – Pseudo code

unmark all vertices

choose some starting vertex s

mark s and insert s into tail of list L

while L is nonempty do

begin

remove a vertex v from **front of L**

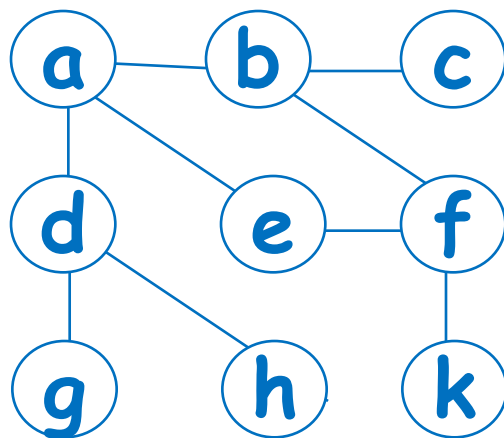
visit v

for each **unmarked neighbor w** of v do

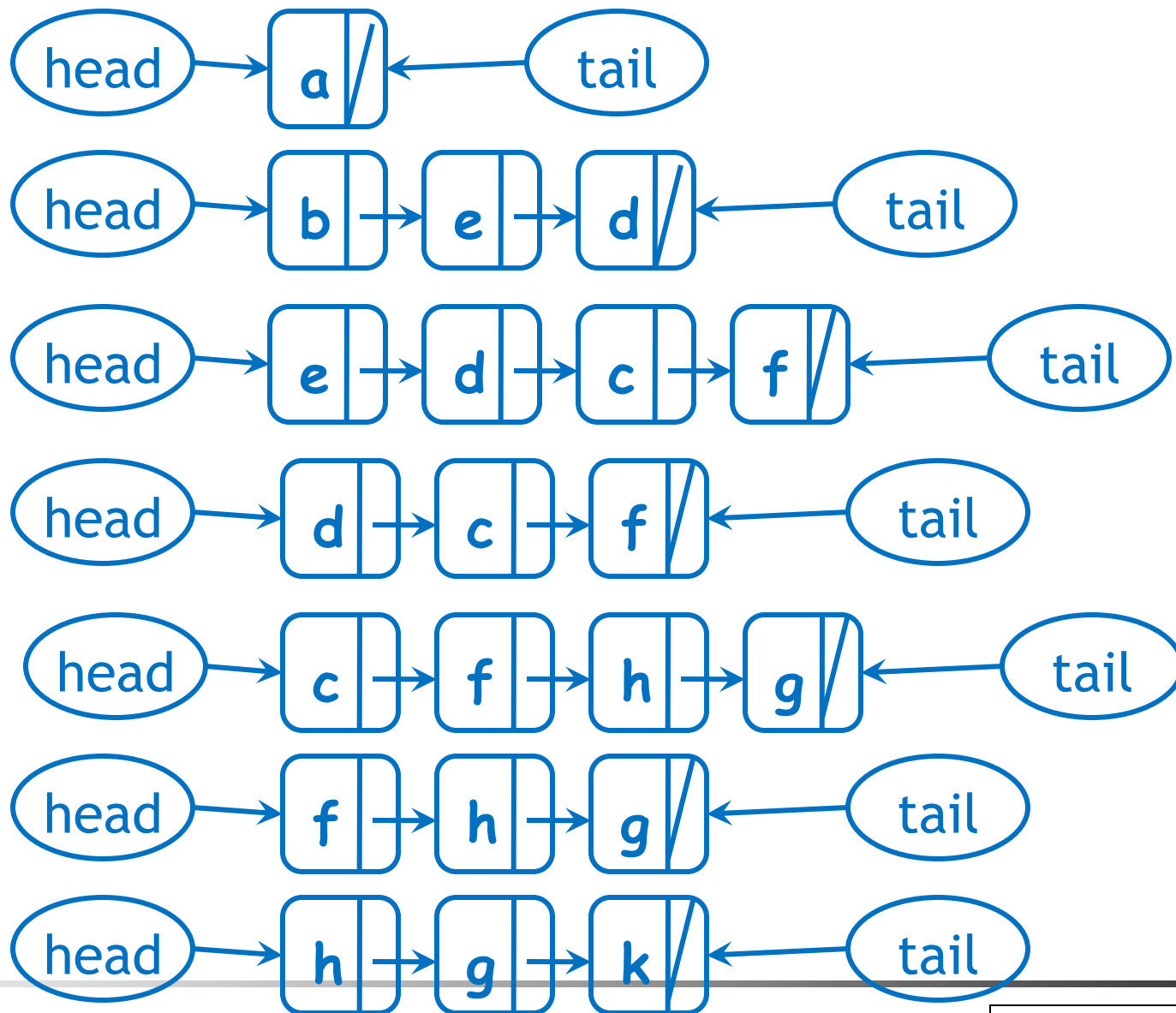
mark w and insert w into tail of list L

end

BFS



a, b, e, d, c, f, h, g, k



& so on ...

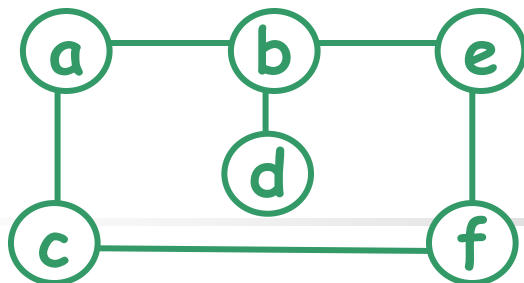
BFS implementation

```
def bfs(graph, start):  
    # Initialize an empty list to store the order of visited vertices  
    visited = []  
  
    # Initialize the queue with the starting vertex  
    queue = [start]  
  
    # Loop until there are no more vertices to visit  
    while queue:  
        # Remove and get the first vertex from the queue (FIFO)  
        vertex = queue.pop(0)  
  
        # If the vertex hasn't been visited yet  
        if vertex not in visited:  
            # Mark it as visited  
            visited.append(vertex)  
  
            # Add all unvisited neighbors to the queue  
            # Use set difference to avoid re-adding visited nodes  
            queue.extend(list(set(graph[vertex]) - set(visited)))  
  
    # Return the list of visited nodes in BFS order  
    return visited  
  
# Example usage: perform BFS starting from vertex 'a'  
bfs(G, "a")
```

```
G = {  
    "a" : ["b", "c"],  
    "b" : ["a", "d"],  
    "c" : ["a", "d"],  
    "d" : ["e"],  
    "e" : ["a"]  
}
```

BFS Task: Find Shortest Path

- BFS explores a graph level by level and is ideal for finding the shortest path in unweighted graphs.
- Given a graph and a start and end node, find the shortest path between them using BFS.
- BFS ensures the first time a node is visited, it is via the shortest path.



Shortest path from a to f:
a - c - f

BFS for Shortest Path – How It Works

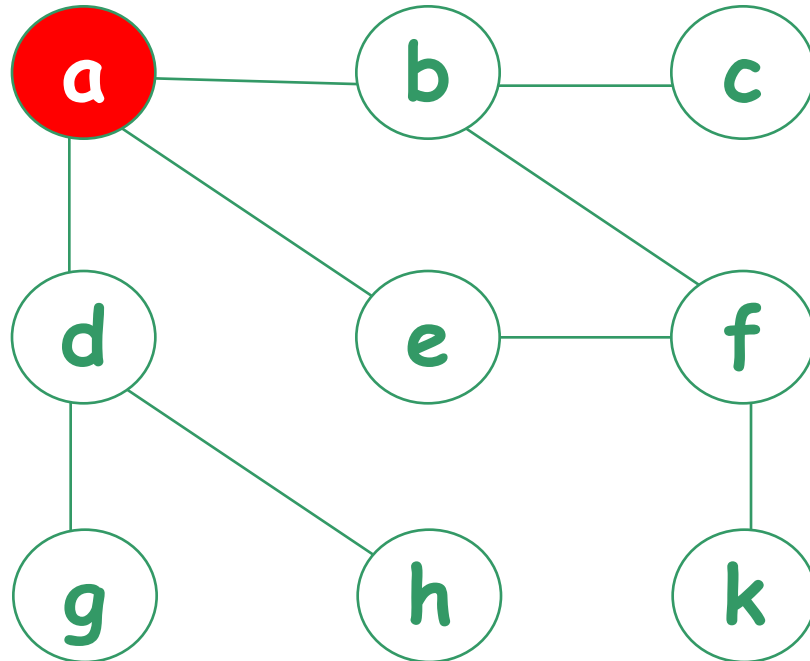
1. Use a queue to explore the graph level-by-level.
 2. Keep track of visited nodes to avoid loops.
 3. Maintain a dictionary to remember the parent of each node.
 4. Reconstruct the path from end to start using the parent map.
- Time complexity: $O(V+E)$
 - Exercise: implement the algorithm in python

Depth First Search (DFS)

Depth First Search (DFS)

- Edges are explored from the most recently discovered vertex, backtracks when finished

The source is a.



Order of exploration

a,

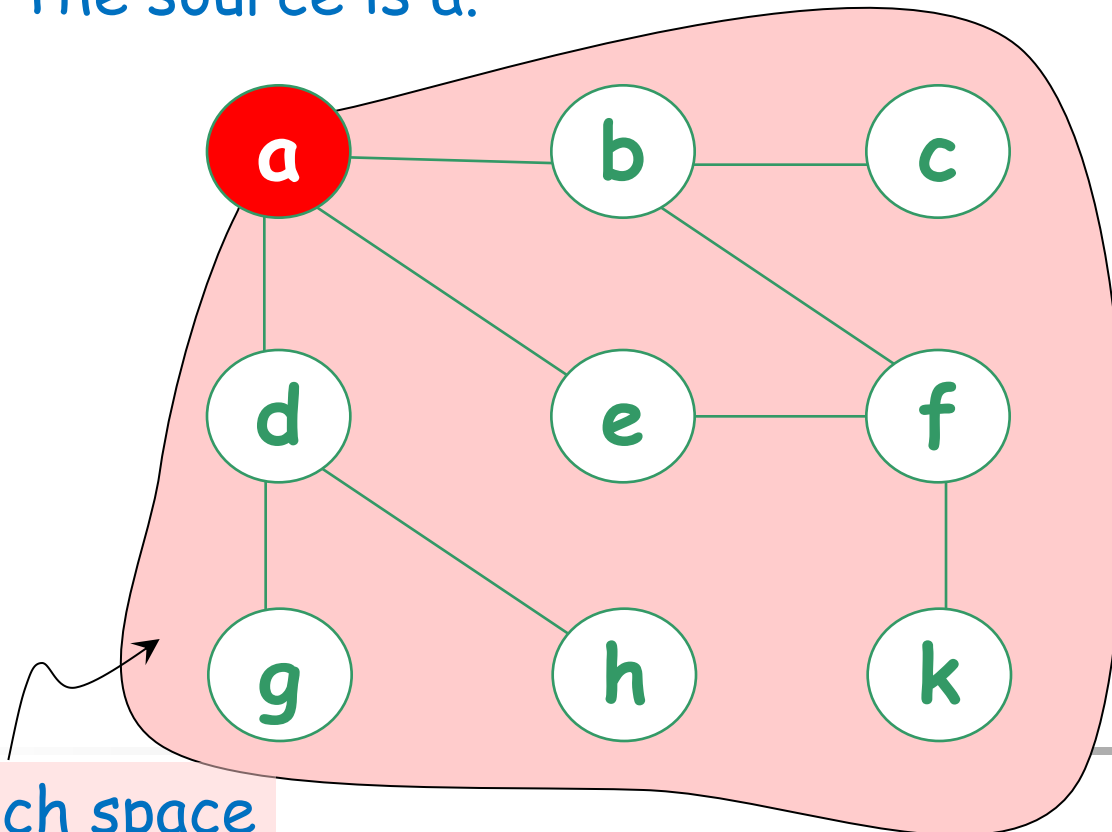
DFS searches
"deeper" in the
graph whenever
possible

Depth First Search (DFS)

- Edges are explored from the most recently discovered vertex, backtracks when finished

The source is a.

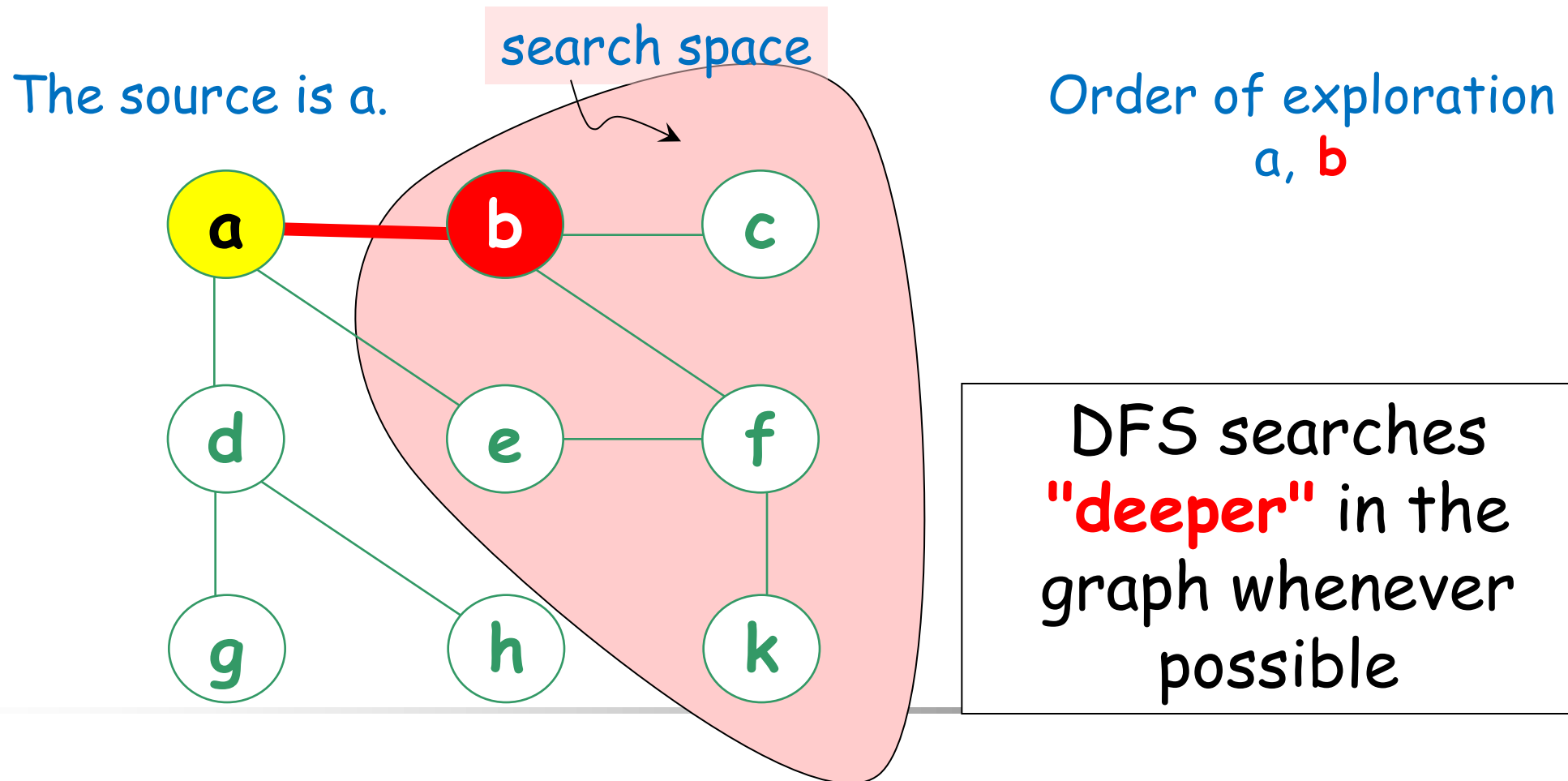
Order of exploration
a,



DFS searches
"deeper" in the
graph whenever
possible

Depth First Search (DFS)

- Edges are explored from the most recently discovered vertex, backtracks when finished



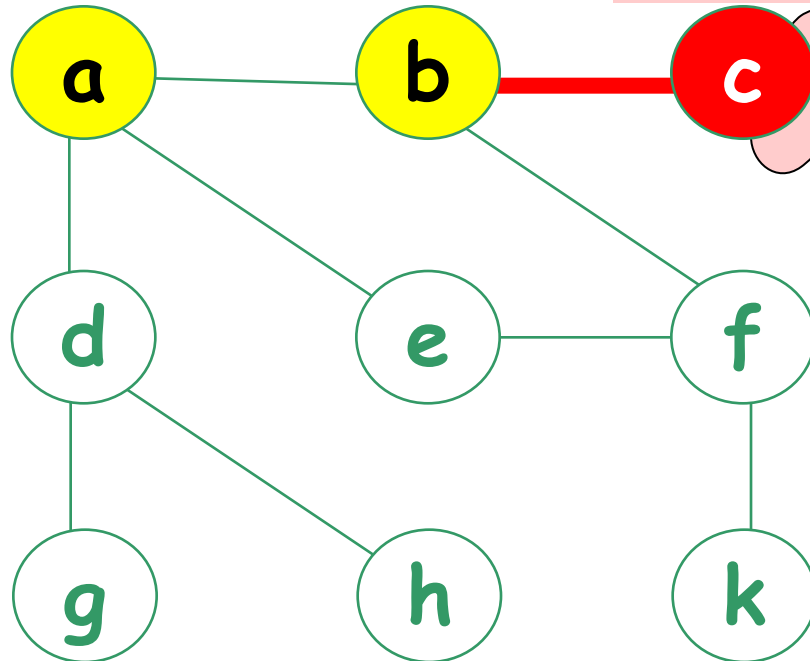
Depth First Search (DFS)

- Edges are explored from the most recently discovered vertex, backtracks when finished

The source is a.

search space
is empty

Order of exploration
a, b, c



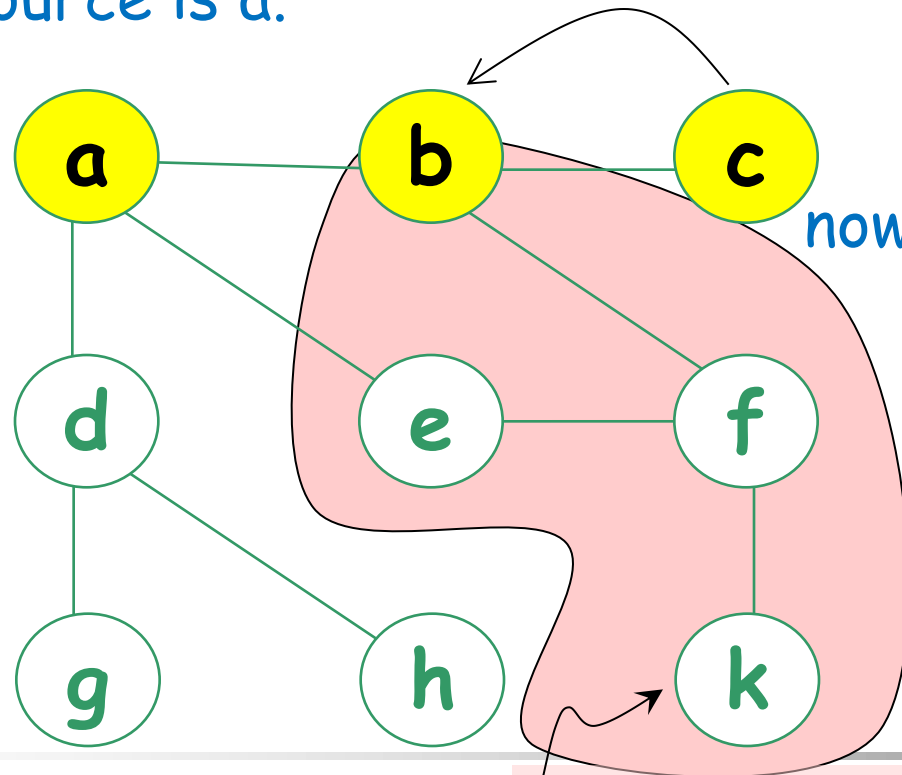
DFS searches
"deeper" in the
graph whenever
possible

Depth First Search (DFS)

- Edges are explored from the most recently discovered vertex, backtracks when finished

The source is a.

Order of exploration
a, b, c



DFS searches
"deeper" in the
graph whenever
possible

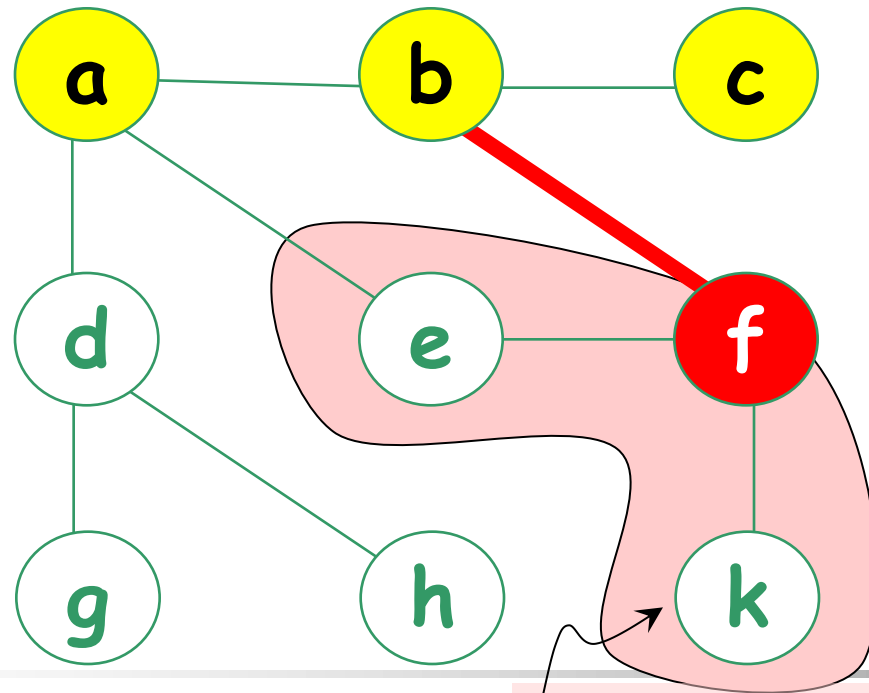
search space

Depth First Search (DFS)

- Edges are explored from the most recently discovered vertex, backtracks when finished

The source is a.

Order of exploration
a, b, c, **f**



DFS searches
"deeper" in the
graph whenever
possible

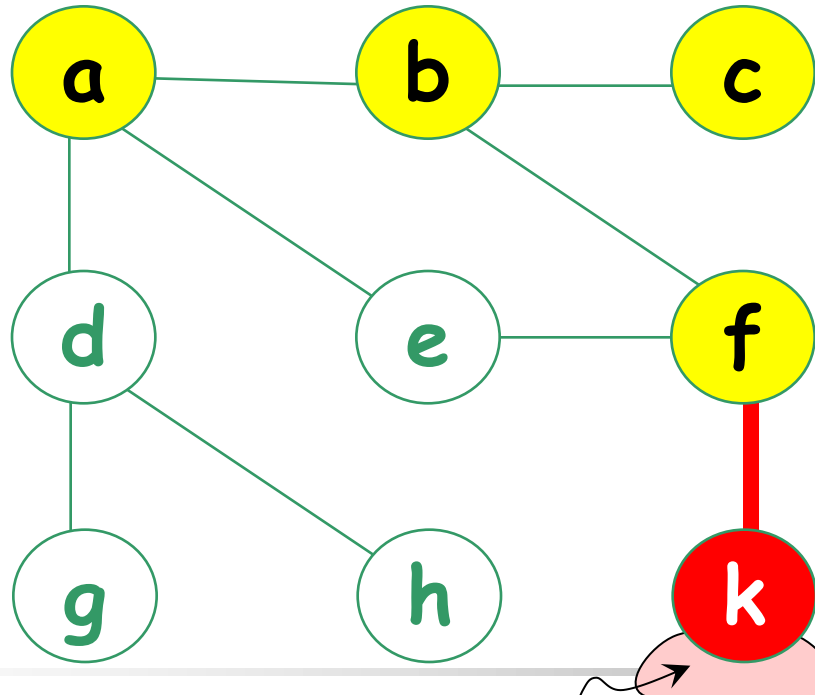
search space

Depth First Search (DFS)

- Edges are explored from the most recently discovered vertex, backtracks when finished

The source is a.

Order of exploration
a, b, c, f, **k**



DFS searches
"deeper" in the
graph whenever
possible

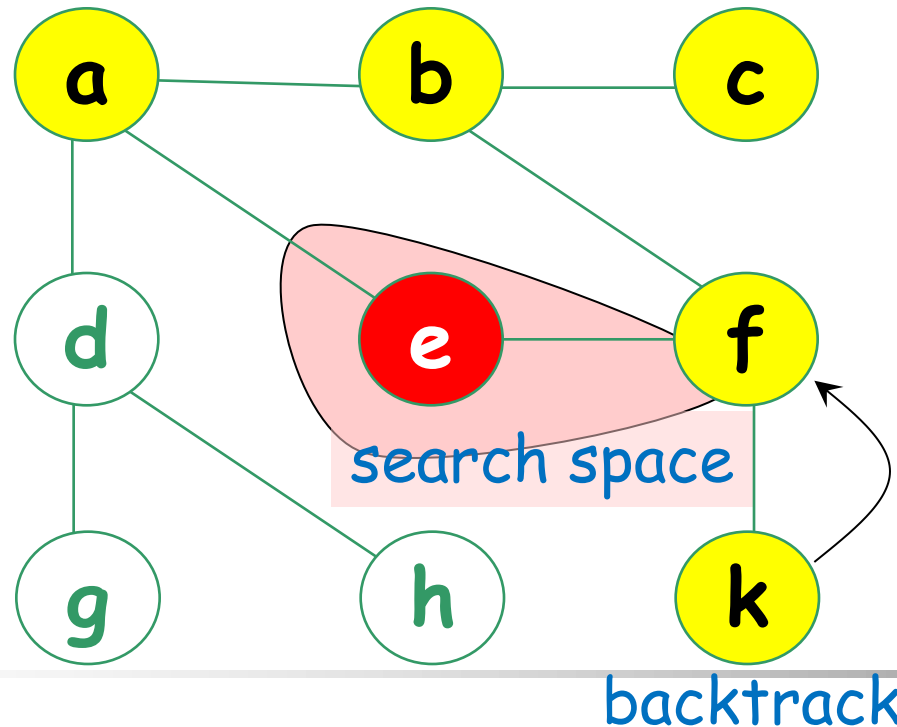
search space is empty

Depth First Search (DFS)

- Edges are explored from the most recently discovered vertex, backtracks when finished

The source is a.

Order of exploration
a, b, c, f, k, **e**



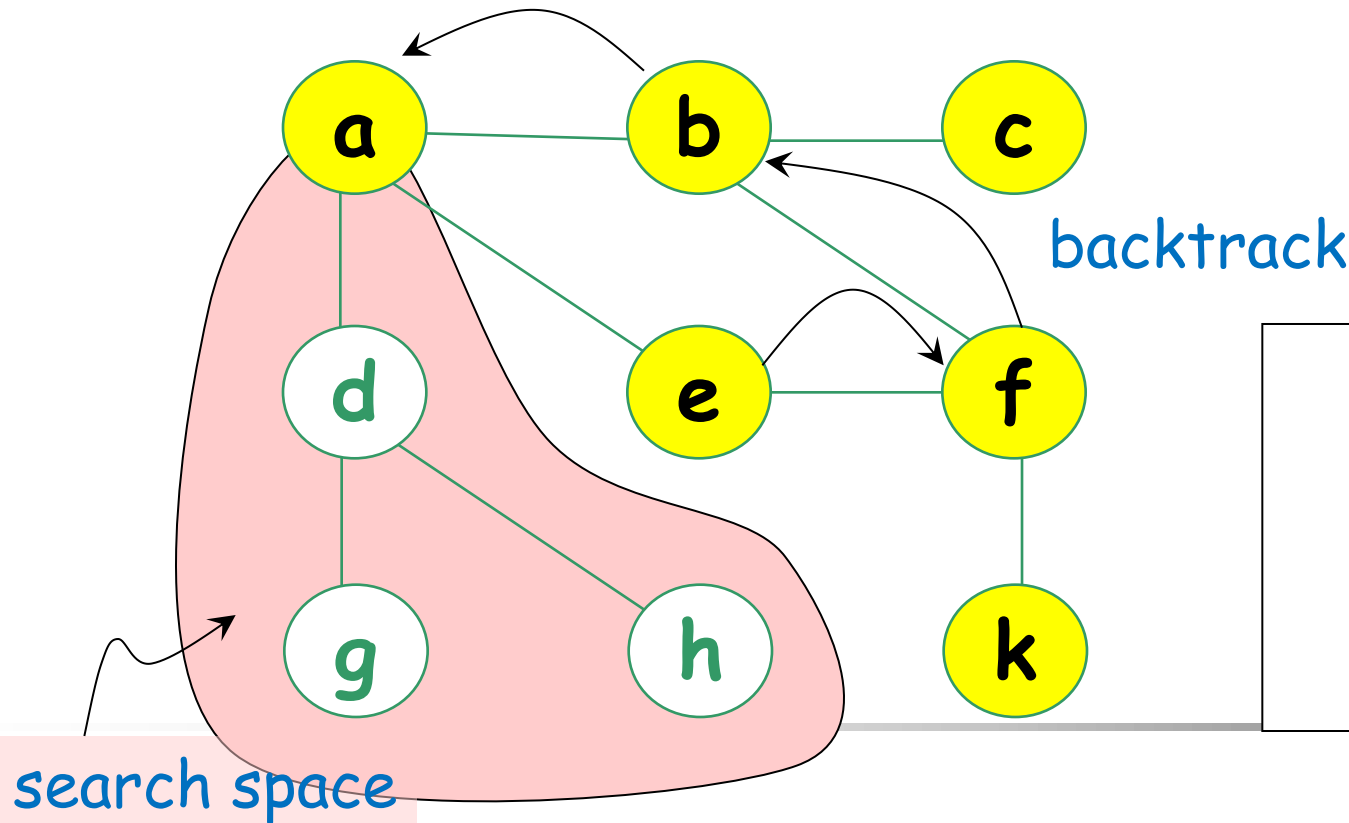
DFS searches
"deeper" in the
graph whenever
possible

Depth First Search (DFS)

- Edges are explored from the most recently discovered vertex, backtracks when finished

The source is a.

Order of exploration
a, b, c, f, k, e



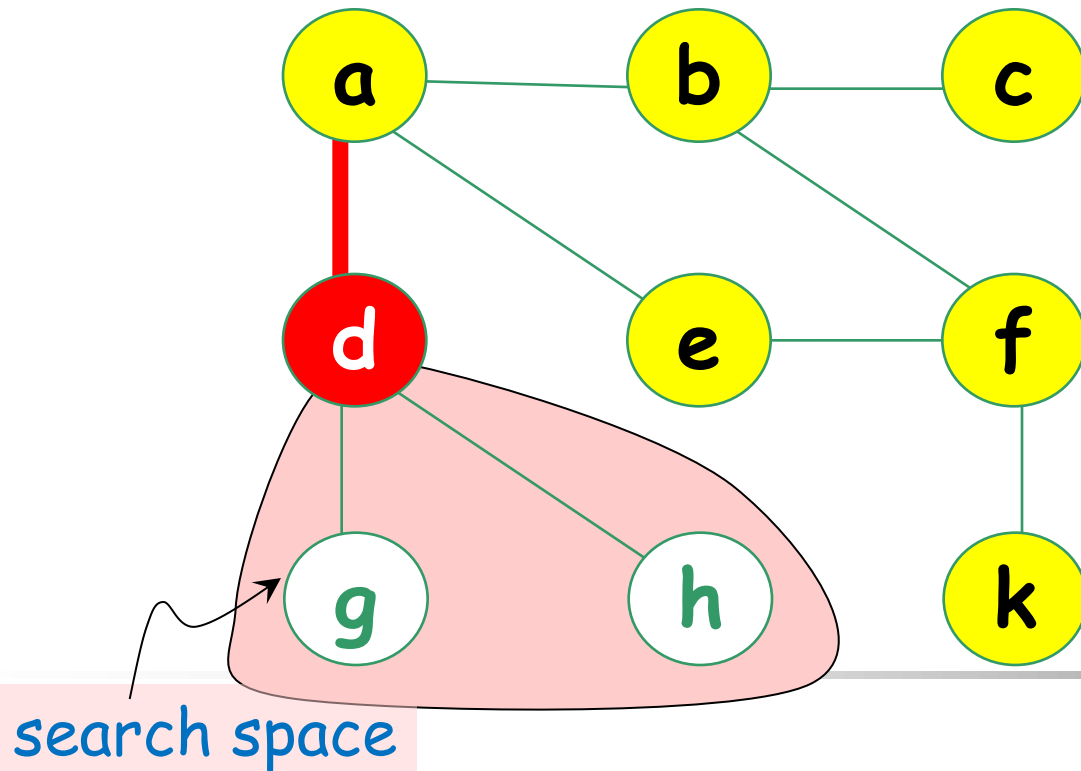
DFS searches
"deeper" in the
graph whenever
possible

Depth First Search (DFS)

- Edges are explored from the most recently discovered vertex, backtracks when finished

The source is a.

Order of exploration
a, b, c, f, k, e, **d**



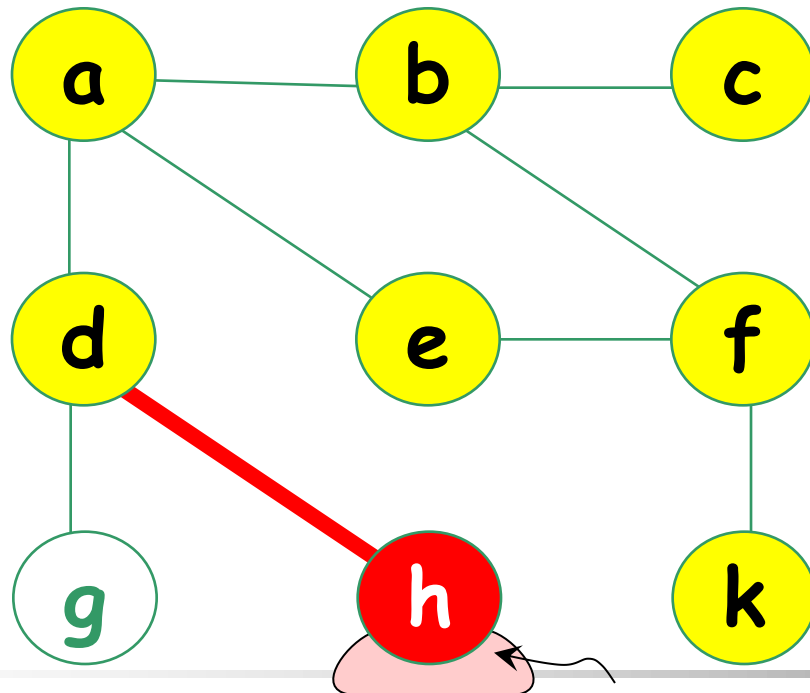
DFS searches
"deeper" in the
graph whenever
possible

Depth First Search (DFS)

- Edges are explored from the most recently discovered vertex, backtracks when finished

The source is a.

Order of exploration
a, b, c, f, k, e, d, **h**



DFS searches
"deeper" in the
graph whenever
possible

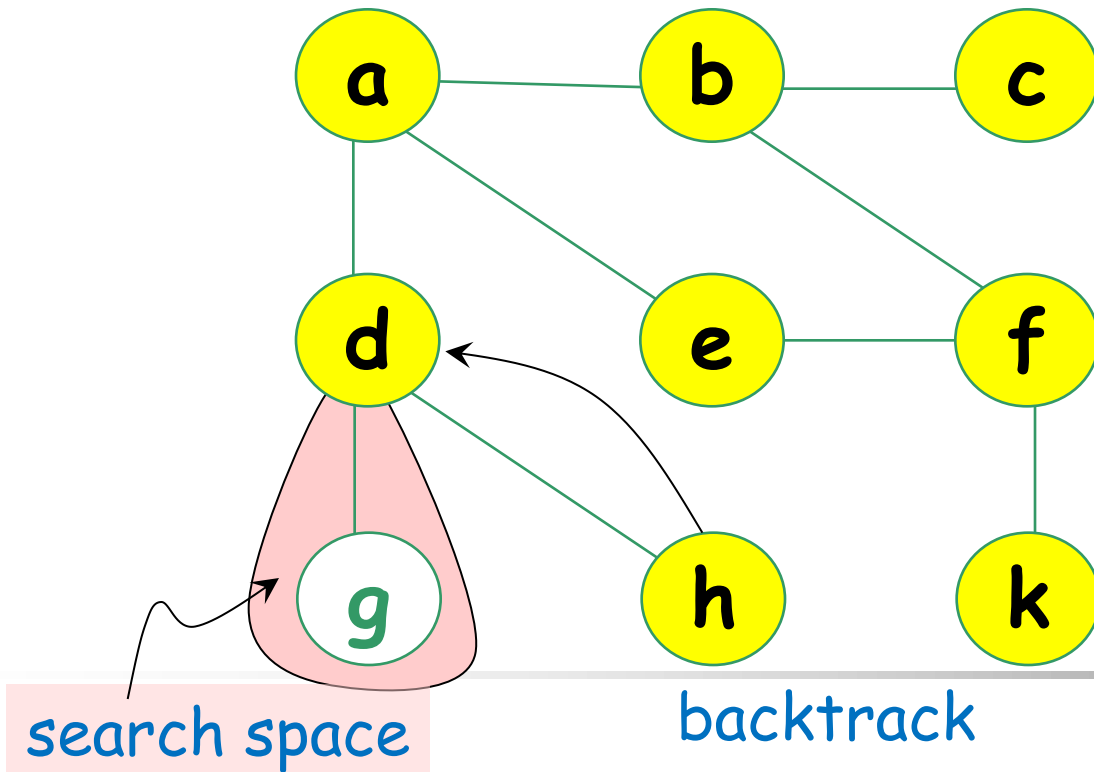
search space is empty

Depth First Search (DFS)

- Edges are explored from the most recently discovered vertex, backtracks when finished

The source is a.

Order of exploration
a, b, c, f, k, e, d, h



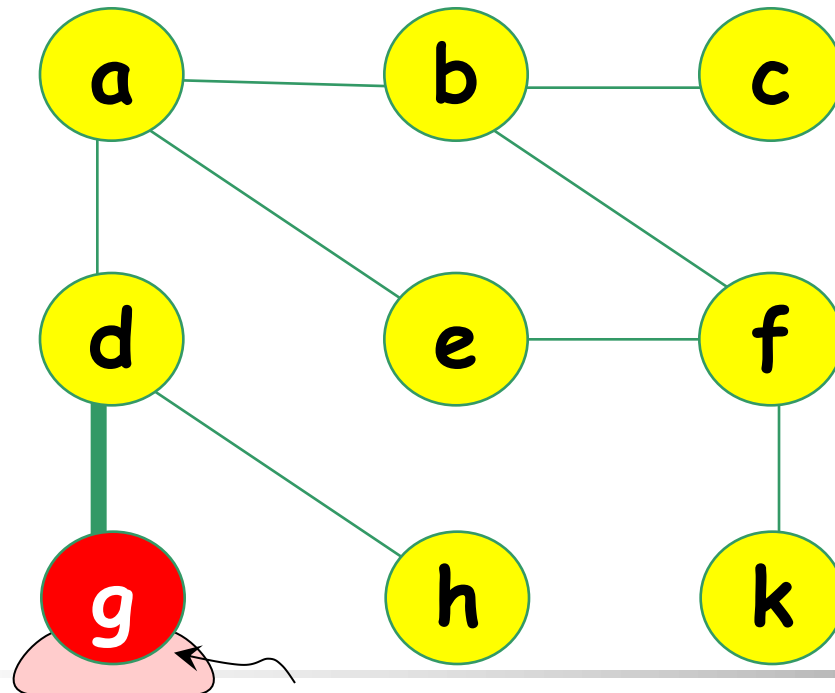
DFS searches
"deeper" in the
graph whenever
possible

Depth First Search (DFS)

- Edges are explored from the most recently discovered vertex, backtracks when finished

The source is a.

Order of exploration
a, b, c, f, k, e, d, h, **g**



search space is empty

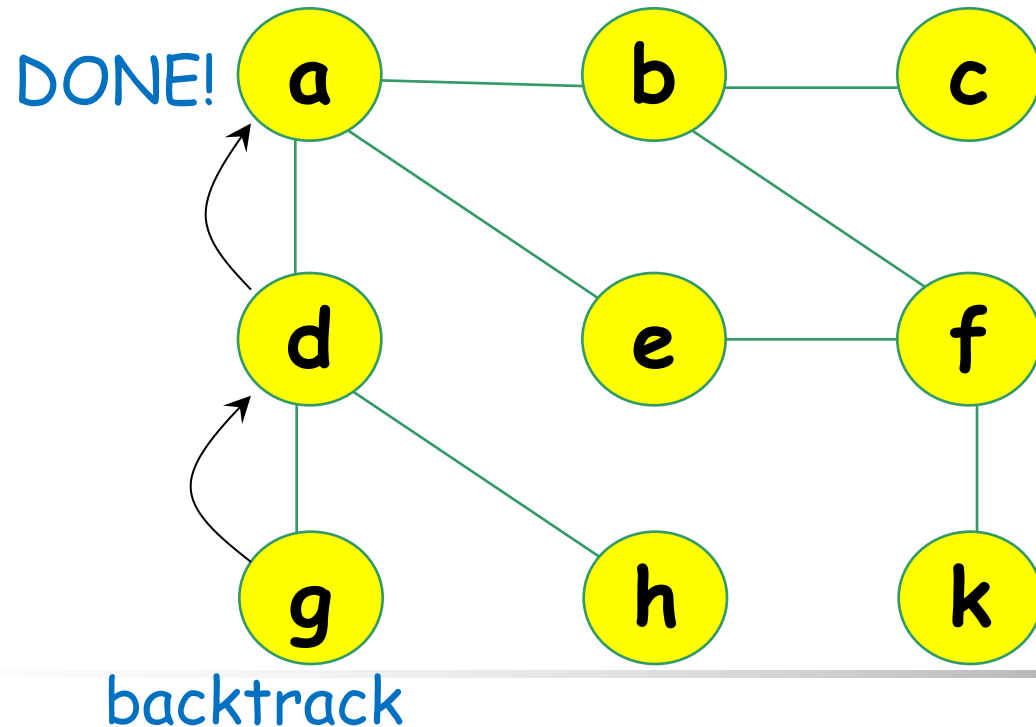
DFS searches
"deeper" in the
graph whenever
possible

Depth First Search (DFS)

- Edges are explored from the most recently discovered vertex, backtracks when finished

The source is a.

Order of exploration
a, b, c, f, k, e, d, h, g



DFS searches
"deeper" in the
graph whenever
possible

Depth First Search (DFS)

- Depth-first search is another strategy for exploring a graph; it search "**deeper**" in the graph whenever possible.
 - Edges are explored from the most recently discovered vertex **v** that still has unexplored edges leaving it.
 - When all edges of **v** have been explored, the search "backtracks" to explore edges leaving the vertex from which **v** was discovered.

DFS – Pseudo code (recursive)

Algorithm DFS(vertex v)

 visit v

 for each **unvisited** neighbor w of v do

 begin

 DFS(w)

 end

DFS – Pseudo code (using stack)

unmark all vertices

push starting vertex **u** onto **top of stack S**

while S is nonempty do

begin

pop a vertex v from **top of S**

if (v is unmarked) then

begin

visit and mark v

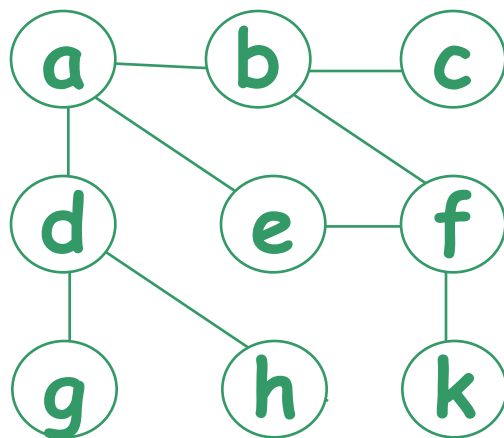
for each **unmarked neighbor w** of v do

push w onto top of S

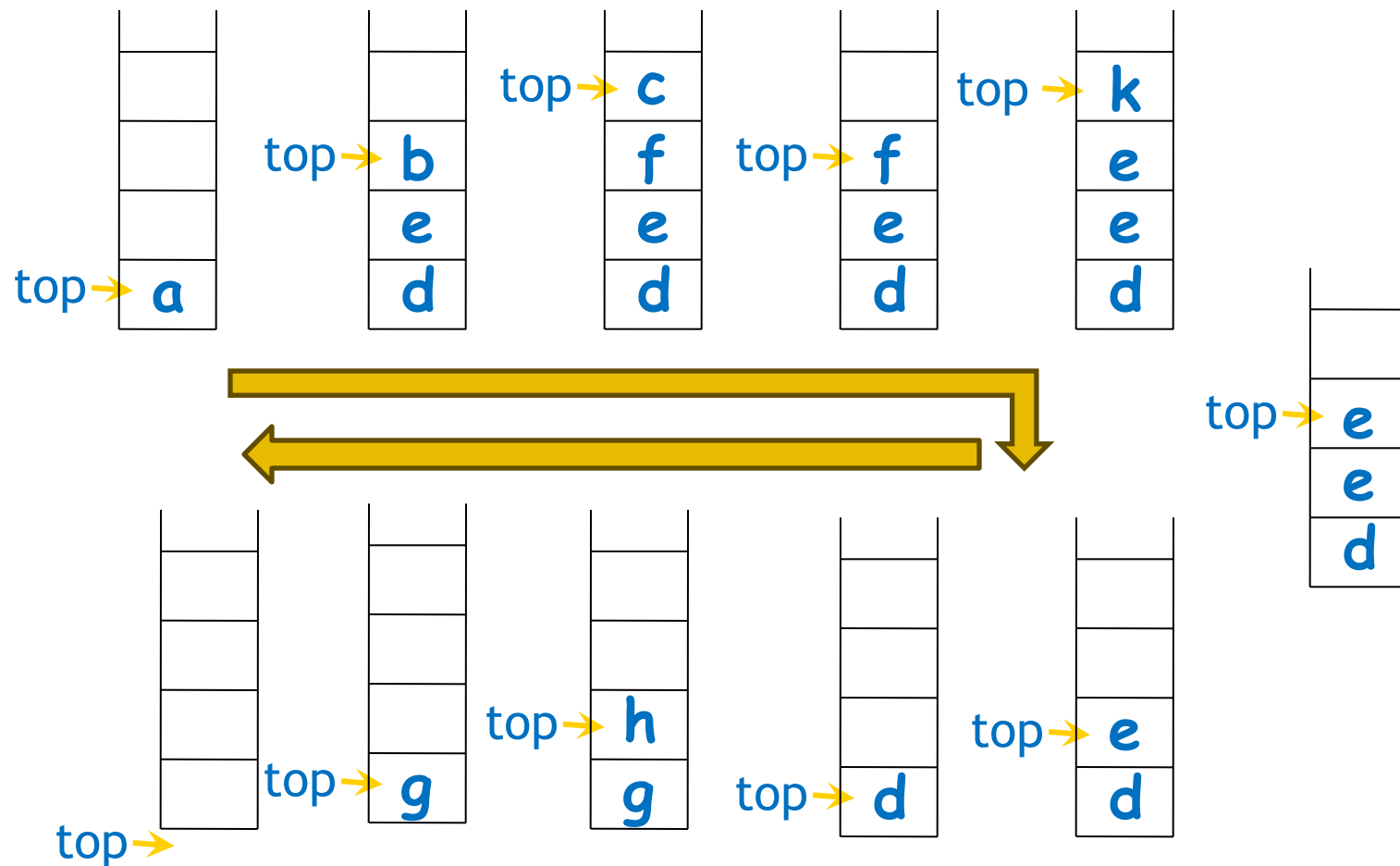
end

end

DFS using Stack



a, b, c, f, k, e, d, h, g



DFS implementation

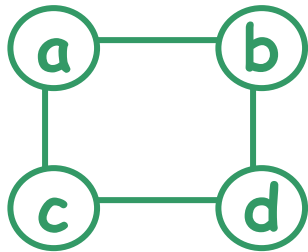
```
def dfs(graph, start):  
    # Initialize an empty list to store the order of visited vertices  
    visited = []  
  
    # Initialize the stack with the starting vertex  
    stack = [start]  
  
    # Loop until there are no more vertices to visit  
    while stack:  
        # Pop the last vertex from the stack (LIFO)  
        vertex = stack.pop()  
  
        # If the vertex hasn't been visited yet  
        if vertex not in visited:  
            # Mark it as visited  
            visited.append(vertex)  
  
            # Add all unvisited neighbors to the stack  
            # Use set difference to avoid re-visiting nodes  
            stack.extend(list(set(graph[vertex]) - set(visited)))  
  
    # Return the list of visited nodes in DFS order  
    return visited  
  
# Example usage: perform DFS starting from vertex 'a'  
dfs(G, "a")
```

```
G = {  
    "a" : ["b", "c"],  
    "b" : ["a", "d"],  
    "c" : ["a", "d"],  
    "d" : ["e"],  
    "e" : ["a"]  
}
```

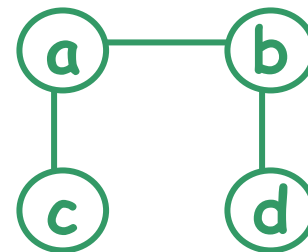
DFS Task:

Detecting Cycles in Undirected Graphs

- A cycle is a path that starts and ends at the same node without repeating edges.
- Important for detecting infinite loops, validating trees, or analyzing dependencies.
- Can be solved using Depth-First Search (DFS)



Example with a cycle



Example without a cycle

DFS-Based Cycle Detection – How It Works

1. Use DFS to explore each vertex.
 2. Keep track of visited nodes.
 3. Track the **parent** of each node to avoid false positives.
 4. If we reach a visited node that's **not the parent**, a cycle is detected.
- Time complexity: $O(V+E)$
 - Exercise: implement the algorithm in python